labeled with RFID tags. But before we find any more things to fill up the memory of the Arduino interface board we need to know more about how that memory works in the first place.

Arduino Memory

The challenging thing with arrays is that they can begin to fill up a relatively enormous amount of memory space. For example, if you look at programming arrays in C for a desktop computer, programmers will often use massive data strings in their arrays to take advantage of all of that RAM and hard-drive memory space. Unlike a computer, our microcontroller has to be programmed differently because it only has a very, very small amount of memory in comparison and arrays take up a lot of that memory space. For example, every letter in a character array takes up one byte of memory so a single 32-character message, like the ones we used in our project code, is going to take up 34 bytes of memory when it is split into two arrays. This can quickly add up and create some problems for us if we don't first know a little bit about how memory is structured on the Arduino microcontroller.

There are three separate types of memory found inside the Atmel ATmega328 microcontroller used on the Arduino Uno interface board. These are shown in Table 8-1.

Table 8-1. *Arduino Memory*

Memory	Size	Storage	Use
Flash	32,768 bytes	non-volatile	Stores the program source code and Arduino bootloader
SRAM	2048 bytes	volatile	Operating space for accessing variables and functions
EEPROM	1024 bytes	non-volatile	Permanent storage for user data like readings or settings

The largest chunk of memory on the microcontroller is flash or program memory. The ATmega328 has a total capacity of 32,768 bytes available to store program instructions, compiled from our source code, with roughly 500 bytes taken up by the Arduino bootloader. The bootloader is a small program that makes programming the microcontroller through a serial port possible rather than requiring additional hardware. Once source code has been uploaded to the microcontroller, its data is read-only and cannot be modified by the running code. It is also a non-volatile memory type meaning that the program stored in this memory is retained even when power is no longer present or the board has been reset.

The program memory is kept in a separate space from the random access memory or RAM—technically this is static or SRAM but it doesn't really matter—that is used for storing data and various microcontroller instructions like keeping track of function calls while the microcontroller is running. The ATmega328 has 2048 bytes of RAM, which gets gobbled up pretty quickly with global variables, arrays, and text strings. This memory type is volatile, so any data in RAM will be lost when the power has been turned off. During a normal operation, an instruction is fetched from the program memory and loaded into RAM to be executed. In this way, local variables are only loaded into RAM for the duration that they are needed while global variables stay in RAM for the entire running time. We will look at some of the problems this can create in a moment.

The final type of memory on the ATmega328 microcontroller is the 1024 bytes of EEPROM; short for electronically erasable program read-only memory. This memory is also non-volatile, keeping its data even when power is not present, although unlike flash memory, EEPROM has a limited life span where it can only be reprogrammed up to about 100,000 times before it becomes unusable. EEPROM is a byte